



Introduction to Git and GitHub

Muhammad Ali

Course : DevOps and Cloud Computing Foundations
Department of Computer Science, SCIT, Beaconhouse National University
(BNU)

Learning Objectives

- At the end of this lesson, you should be able to explain:
 - how git creates a mini-filesystem in your directory
 - what commit, push, pull, and sync do
 - the elements of the basic git workflow
 - how git allows you to work across multiple computers
 - how git allows you and a partner to work together



I KILLED A MAN REWORKED OLD MEME



I manually upload files
to github instead of using git

DUDE.
WTF?!



Git is a **distributed** version-control system

- You keep your files in a *repository* on your local machine.
- You synchronize your repository with a repository on a server.
- If you move from one machine to another, you can pick up the changes by synchronizing with the server.
- If your partner uploads some changes to your files, you can pick those up by synchronizing with the server.
- **Git** is a software like Word's "tracked changes."
- **GitHub** is a collaboration tool like Dropbox or Google Drive.

Why are Git & GitHub useful?

- **Internal management**

- Code is safely stored in one central location
- All project team have access to the same version of code

- **External collaboration**

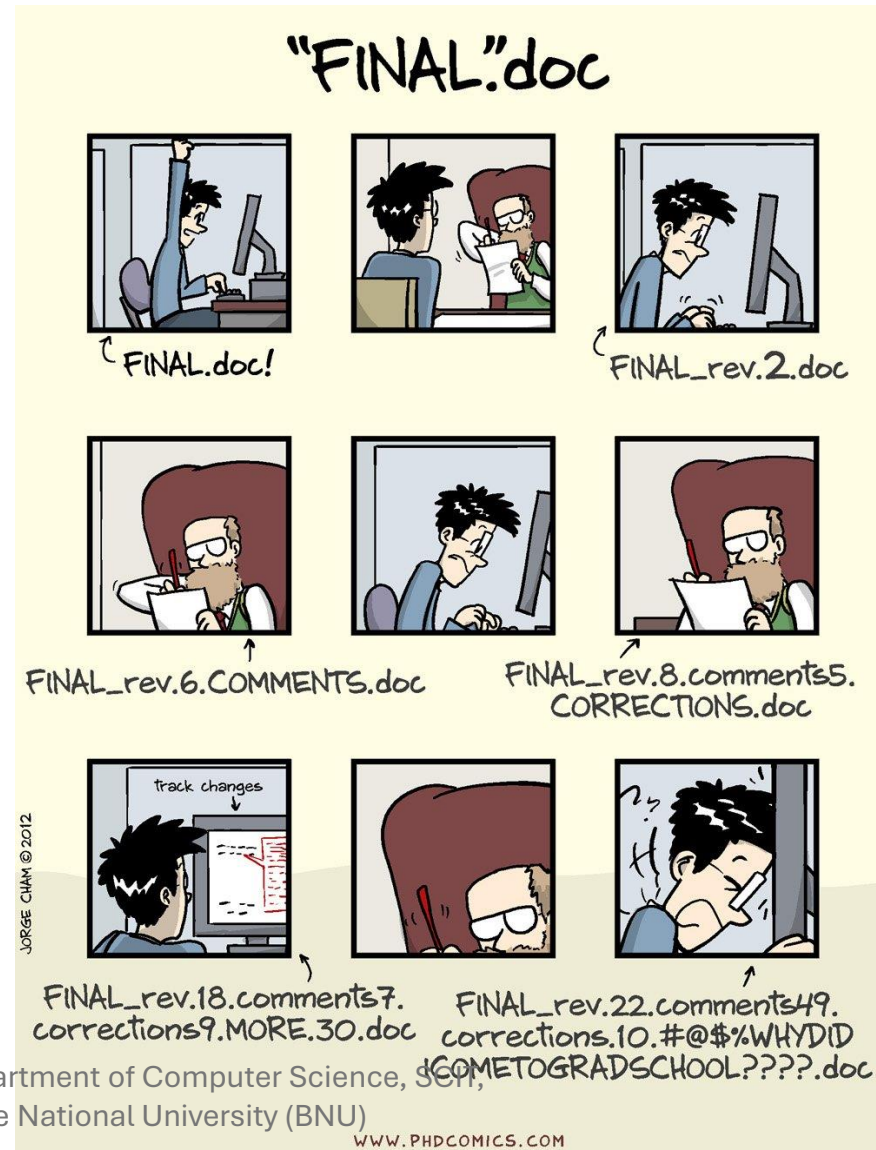
- Share code developed by you
- Work together on cross-dept projects

- **Reproducibility over time**

- Efficiently produce results on a schedule

Why are Git & GitHub useful?

Git & GitHub help avoid this →



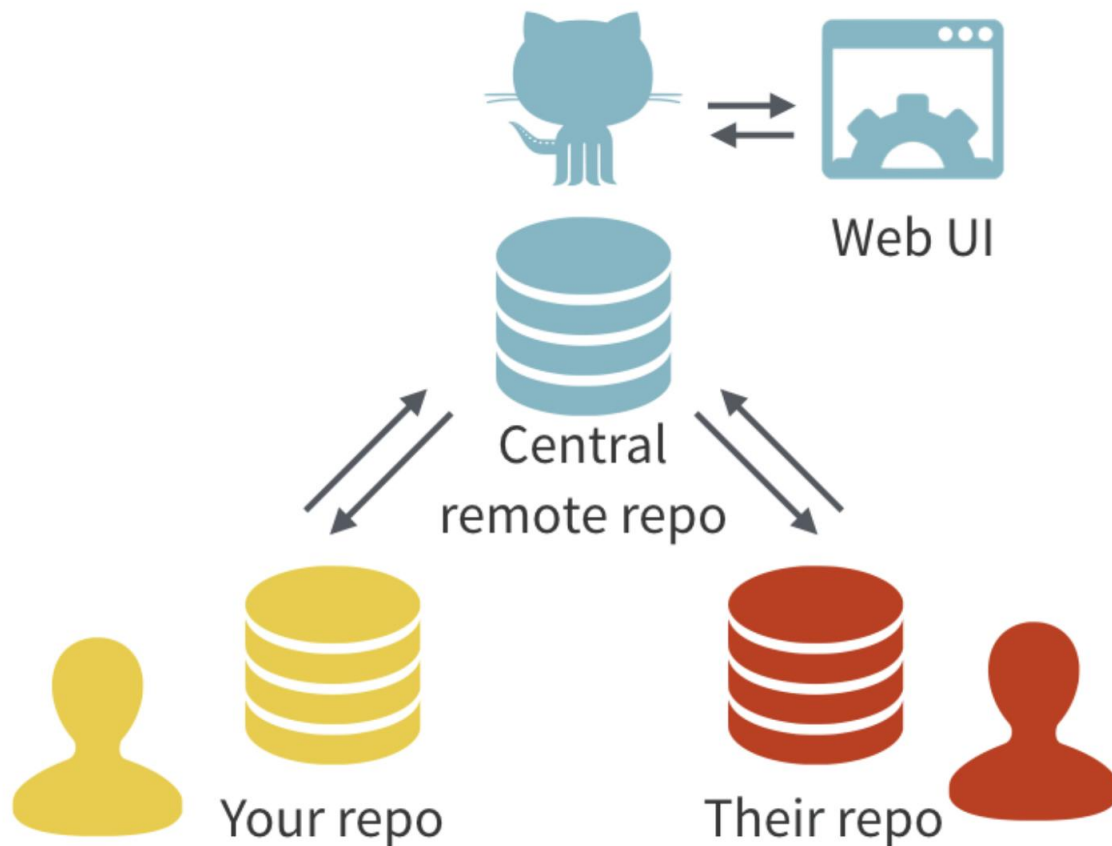
Git is a distributed **version-control** system

- Terminology: In git-speak, a “version” is called a “commit.”
- Git keeps track of the history of your commits, so you can go back and look at earlier versions, or just give up on the current version and go back some earlier version.

Key Terms & Concepts

- **Git** lets you take a snapshot (or **commit**) of files in a folder (or **repository** or **repo**).
 - You decide when to take a snapshot and which files to include.
- **GitHub** lets you back up your work in a **remote** place.
 - To get work on your **local** computer for the first time, you would **clone** a repo.
 - To send updates to GitHub, you would **push** changes.
 - To download updates from GitHub, you would **pull** changes.

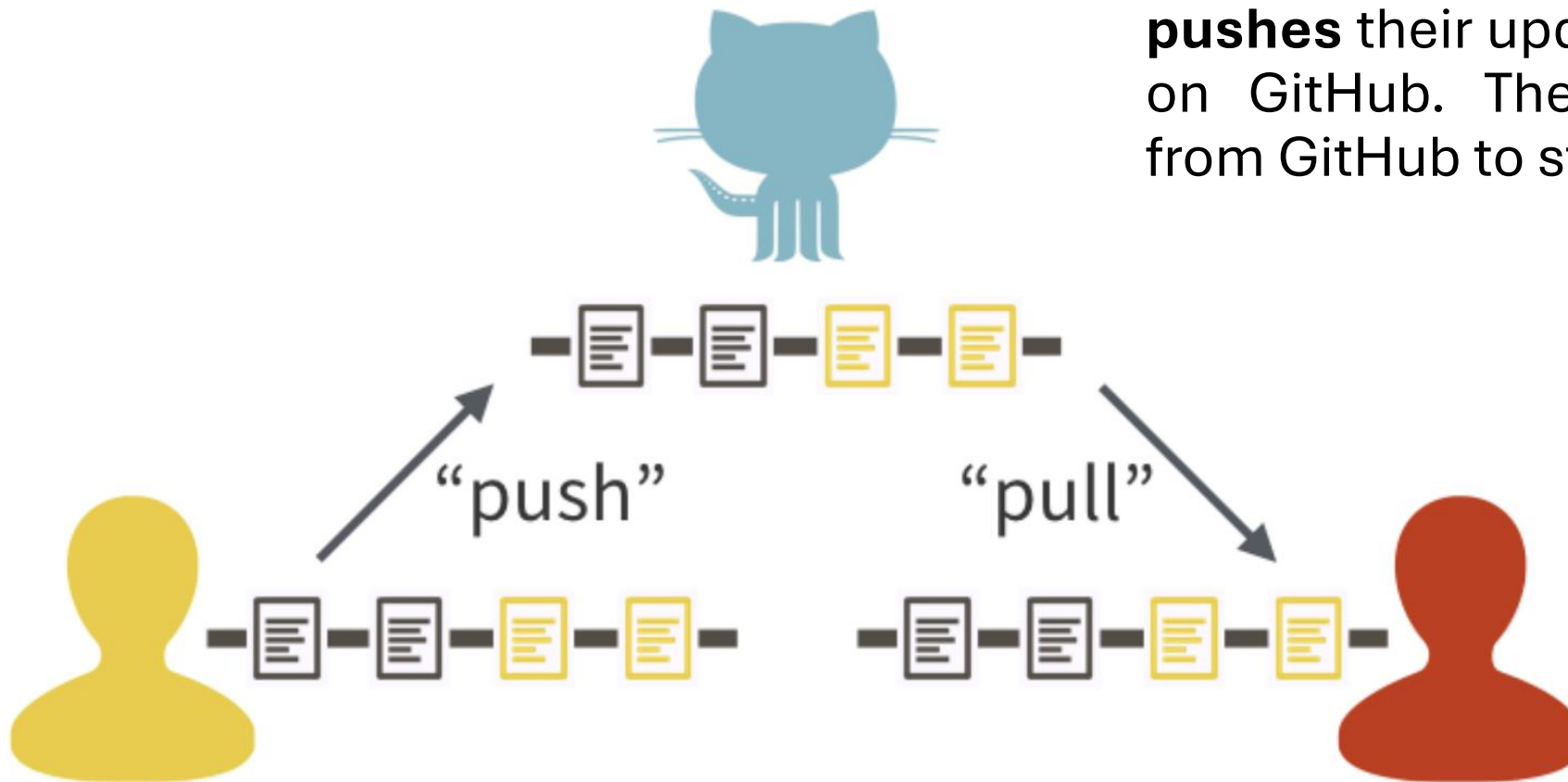
Key Terms & Concepts



All contributors have a **local** copy of the Git repository. These stay in sync through a central **remote** repository hosted on GitHub.

Key Terms & Concepts

One contributor makes **commits** and **pushes** their updates to the central version on GitHub. The other contributor **pulls** from GitHub to stay up-to-date.



**When you're dead but remember you
forgot to git commit git push your
last code iterations**

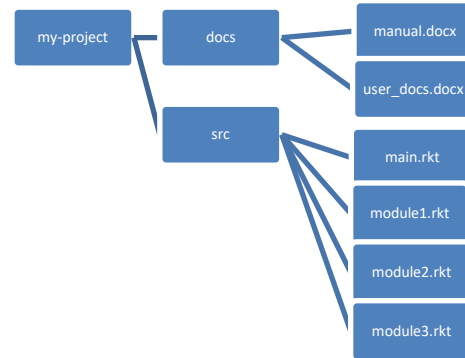


Muhammad Ali - Department of Computer Science, SCIT,
Beaconhouse National University (BNU)

A simple model of git

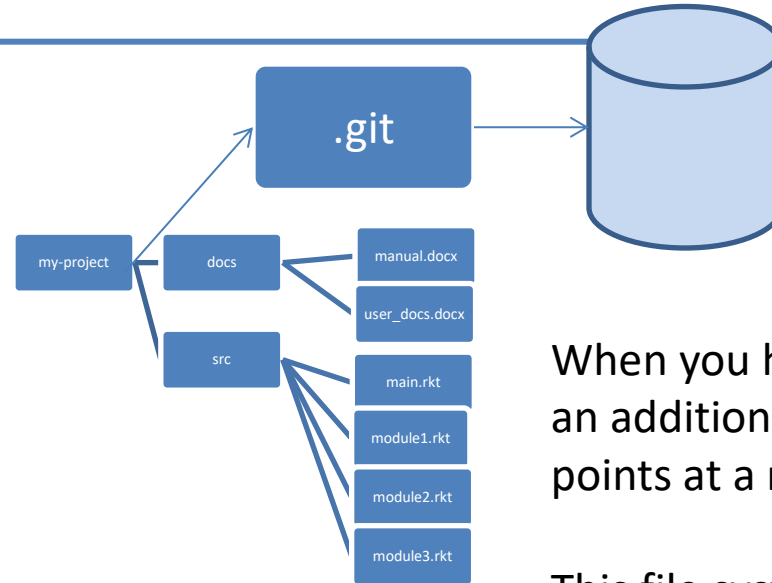
- Most git documentation gets into details very quickly.
- Here's a very simple model of what's going on in git.

Your files



Here are your files, sitting
in a directory called my-
project

Your files in your git repository

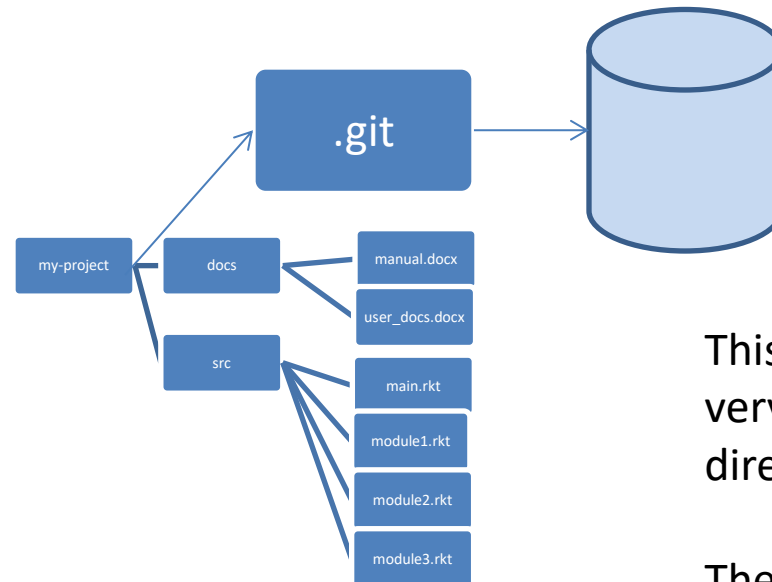


When you have a git repository, you have an additional directory called `.git`, which points at a mini-filesystem.

This file system keeps all your data, plus the bells and whistles that git needs to do its job.

All this sits on your local machine.

The git client



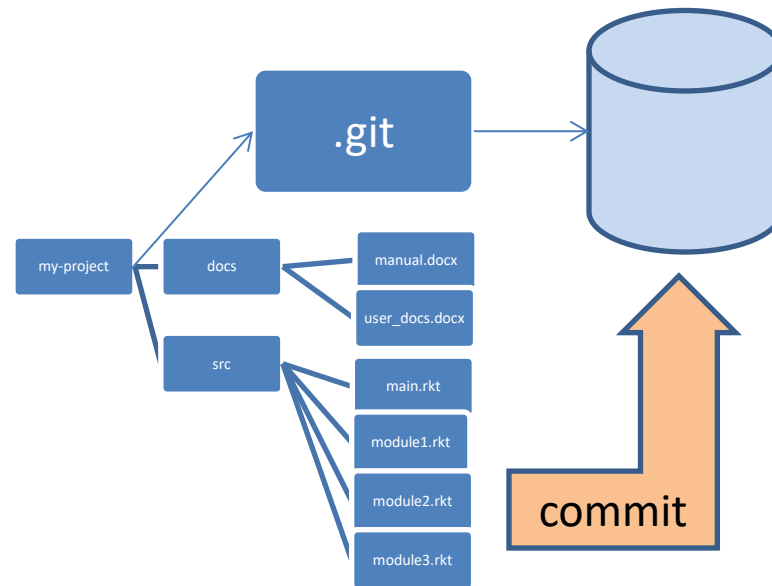
This mini-filesystem is highly optimized and very complicated. Don't try to read it directly.

The job of the git client (either Github for Windows, Github for Mac, or a suite of command-line utilities) is to manage this for you.

Your workflow (part 1)

- You edit your local files directly.
 - You can edit, add files, delete files, etc., using whatever tools you like.
 - This doesn't change the mini-filesystem, so now your mini-fs is behind.

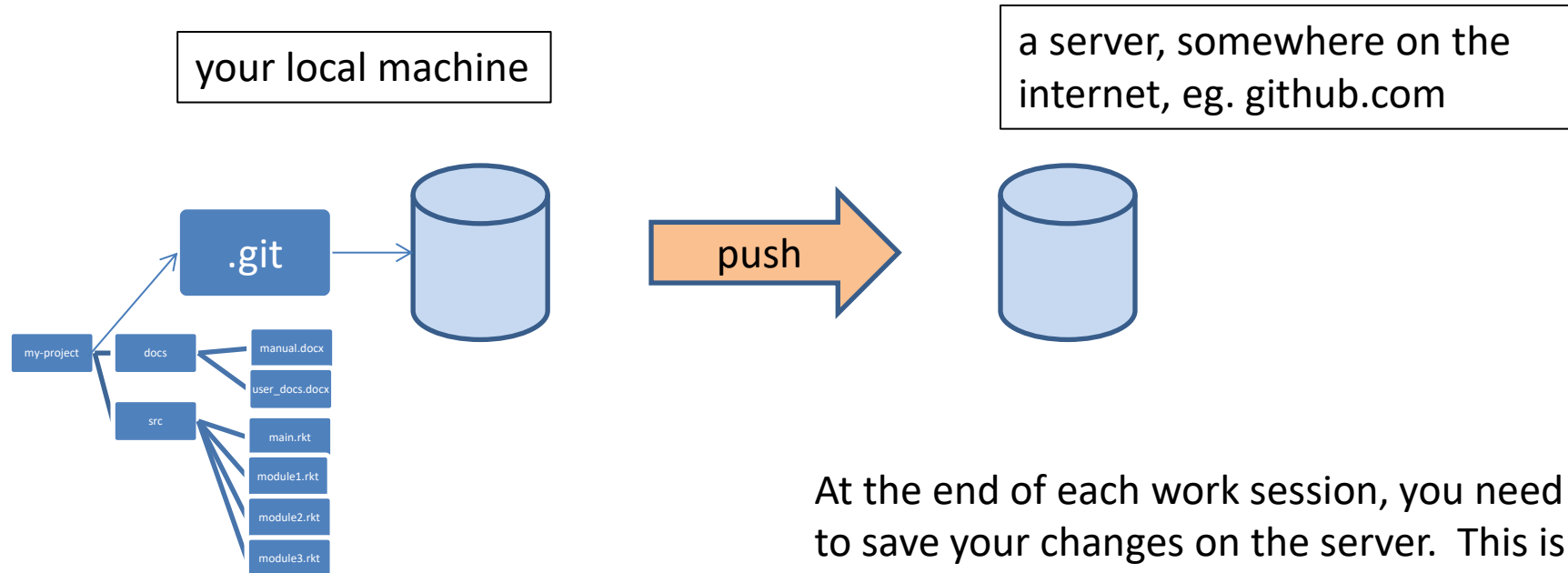
A Commit



When you do a “commit”, you record all your local changes into the mini-fs.

The mini-fs is “append-only”. Nothing is ever over-written there, so everything you ever commit can be recovered.

Synchronizing with the server (1)

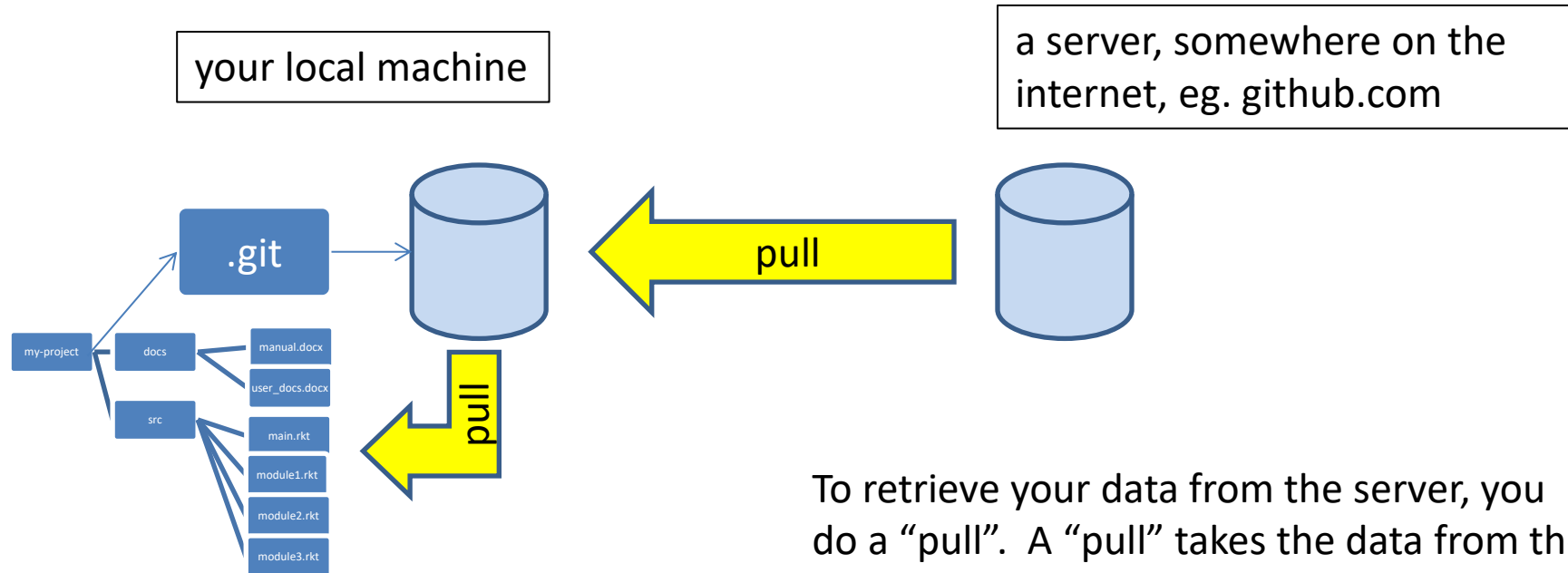


At the end of each work session, you need to save your changes on the server. This is called a “push”.

Now all your data is backed up.

- You can retrieve it, on your machine or some other machine.
- We can retrieve it (that’s how we collect homework)

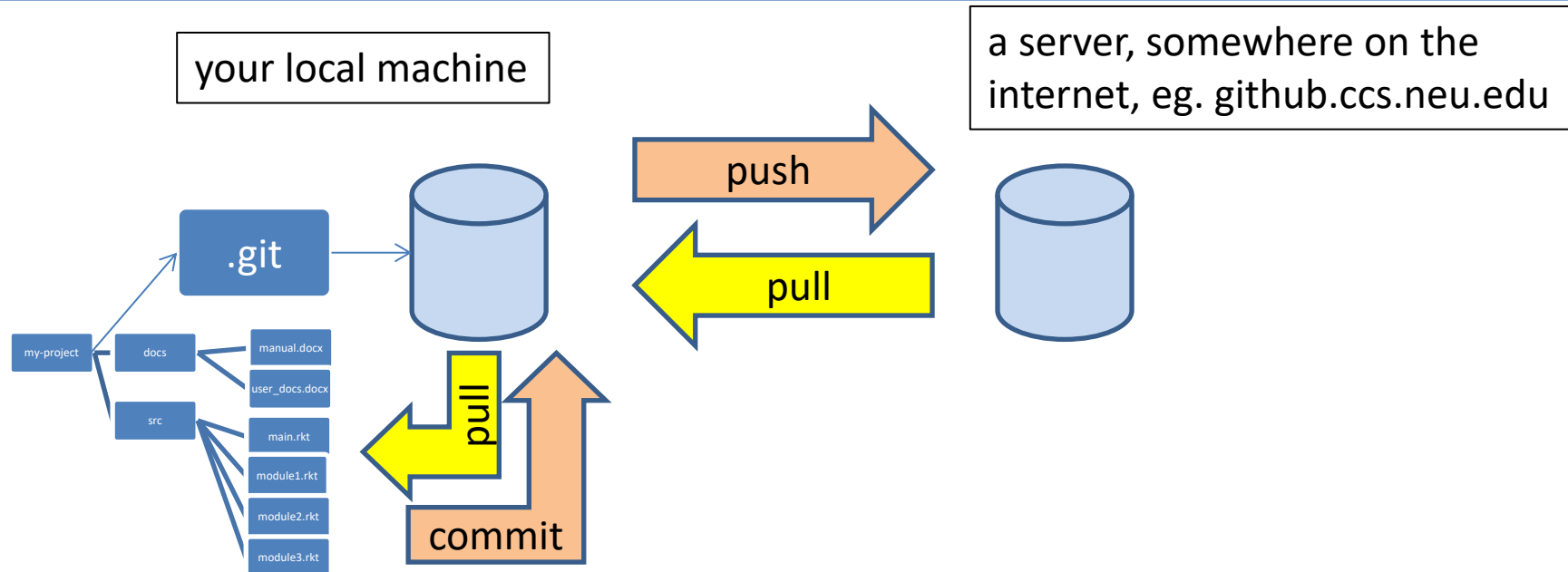
Synchronizing with the server (2)



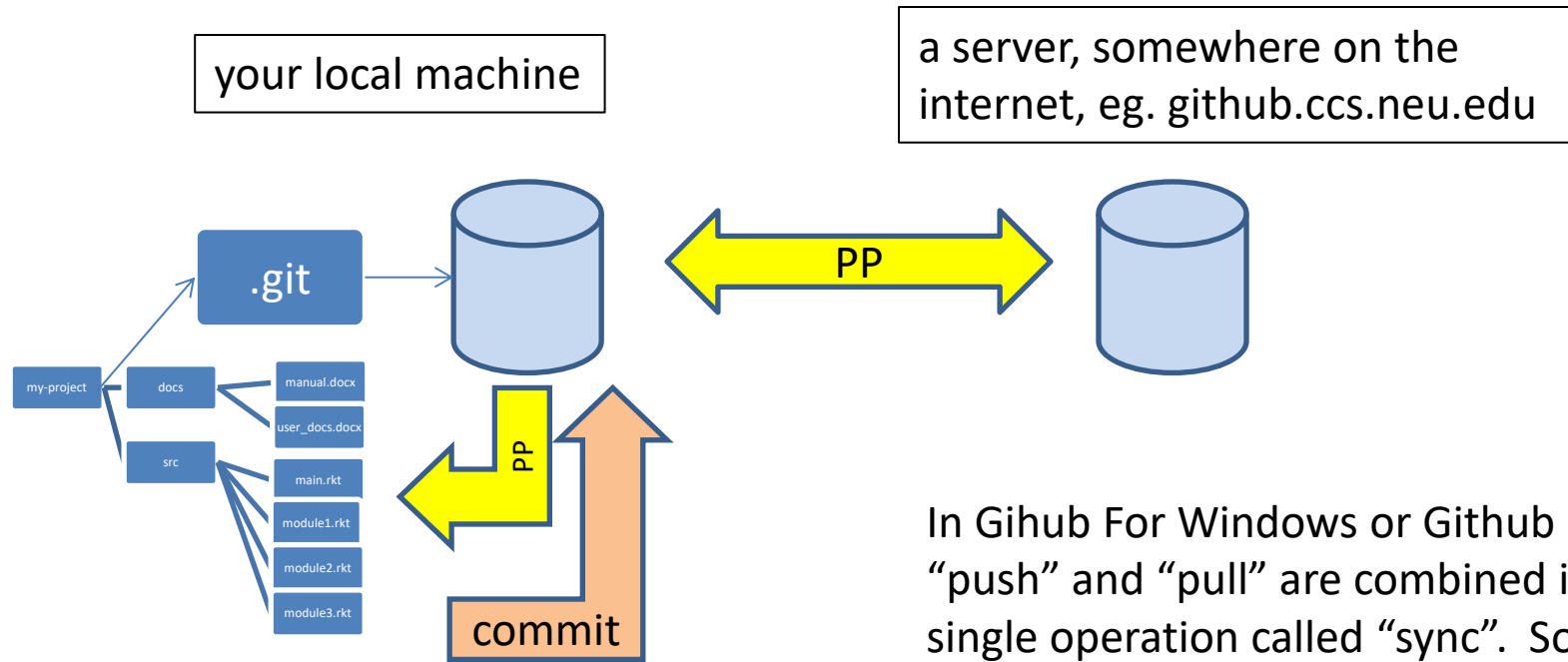
To retrieve your data from the server, you do a “pull”. A “pull” takes the data from the server and puts it both in your local mini-fs and in your ordinary files.

If your local file has changed, git will merge the changes if possible. If it can't figure out how to the merge, you will get an error message. We'll learn how to deal with these in the next lesson.

The whole picture

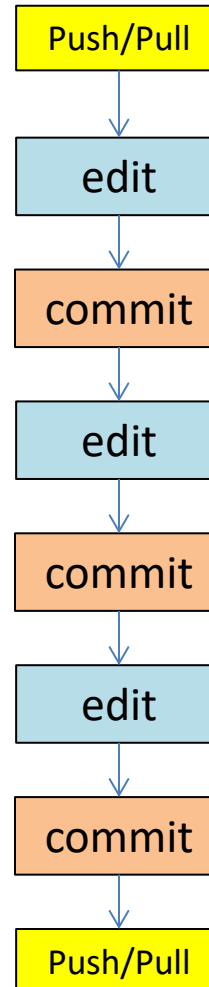


The whole picture using GHFW



In Github For Windows or Github For Mac, “push” and “pull” are combined into a single operation called “sync”. So in these clients, there are only two steps (“commit” and “sync”) to worry about, not three.

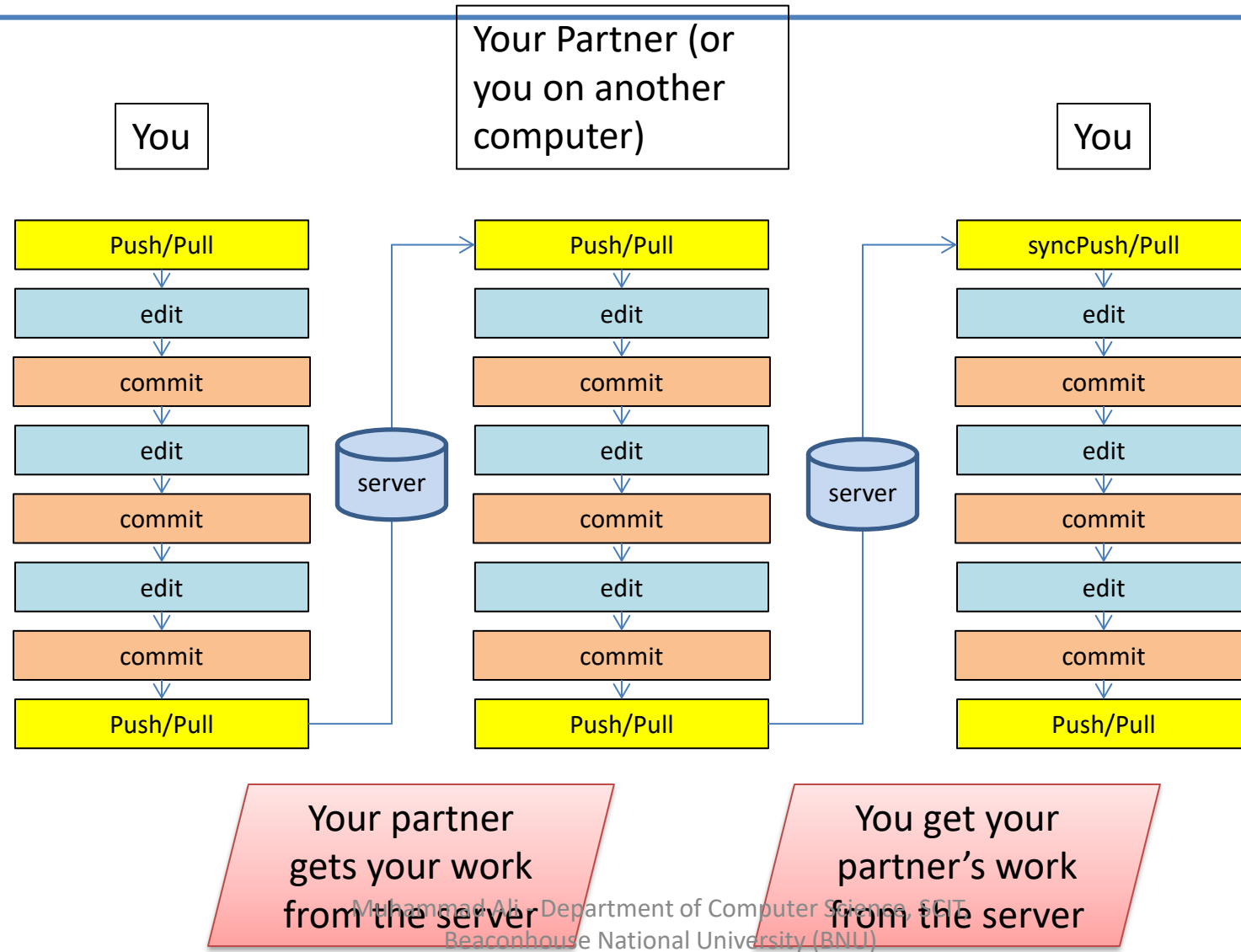
Your workflow (2)



Best practice: commit your work whenever you've gotten one part of your problem working, or before trying something that might fail.

If your new stuff is screwed up, you can always “revert” to your last good commit. (Remember: always “revert”, never “roll back”)

Your workflow with a partner



Advice

**Time and resources don't guarantee doing it right—
deep understanding and rigorous hard work do**



Muhammad Ali - Department of Computer Science, SCIT,
Beaconhouse National University (BNU)

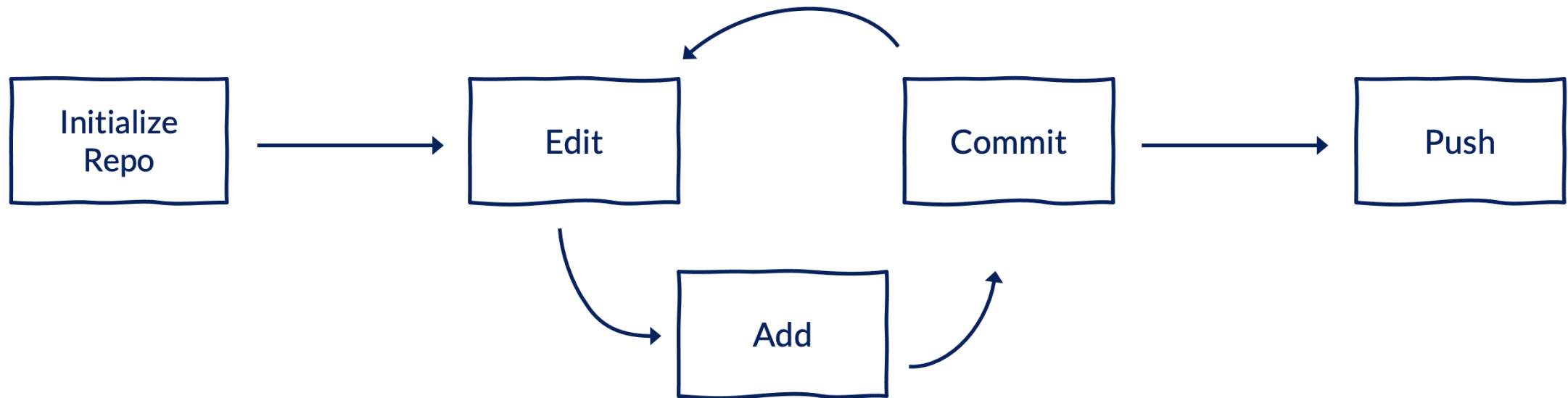
Initial Setup

- Create a GitHub account.
- Install Git and GitHub Desktop on your computer.
- Configure Git and GitHub.
 - Configure your GitHub username and email address.
 - Set up a personal access token (PAT).

See [GitHub's Getting Started guide](#) for detailed instructions.

Basic Workflow

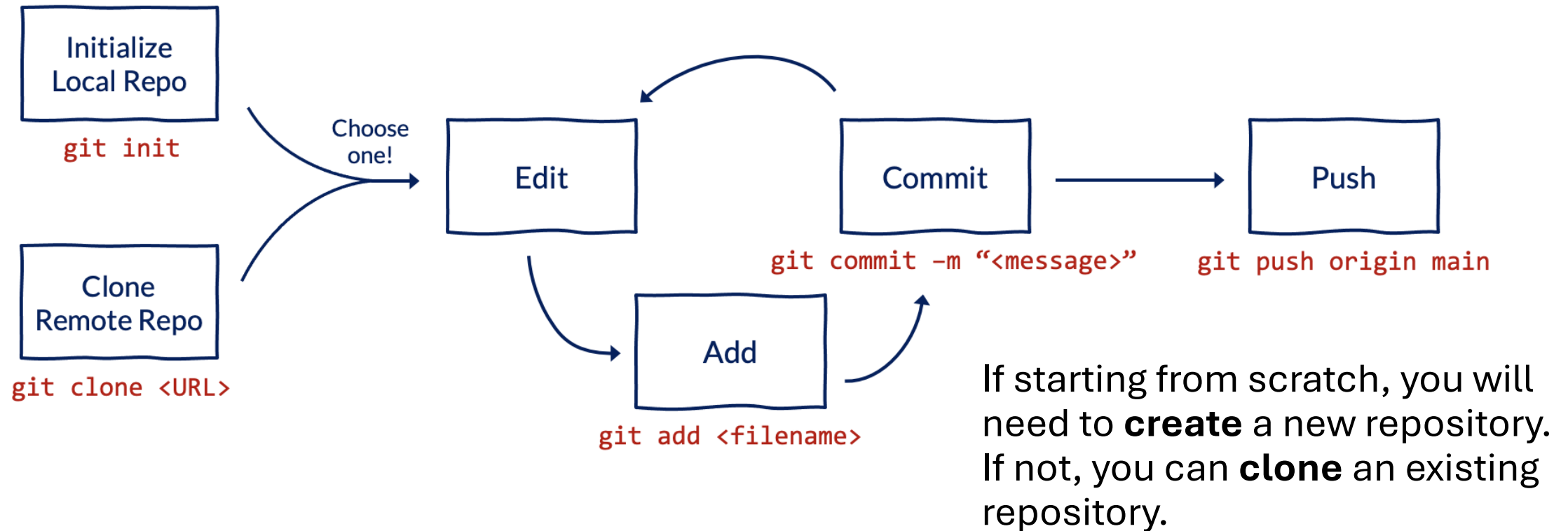
- Initialize a repository (**create** or **clone**), edit files, tell Git which files to track (**add**), take a snapshot of those files (**commit**), send those snapshots to GitHub (**push**), and repeat.



Basic Workflow

LOCAL

REMOTE



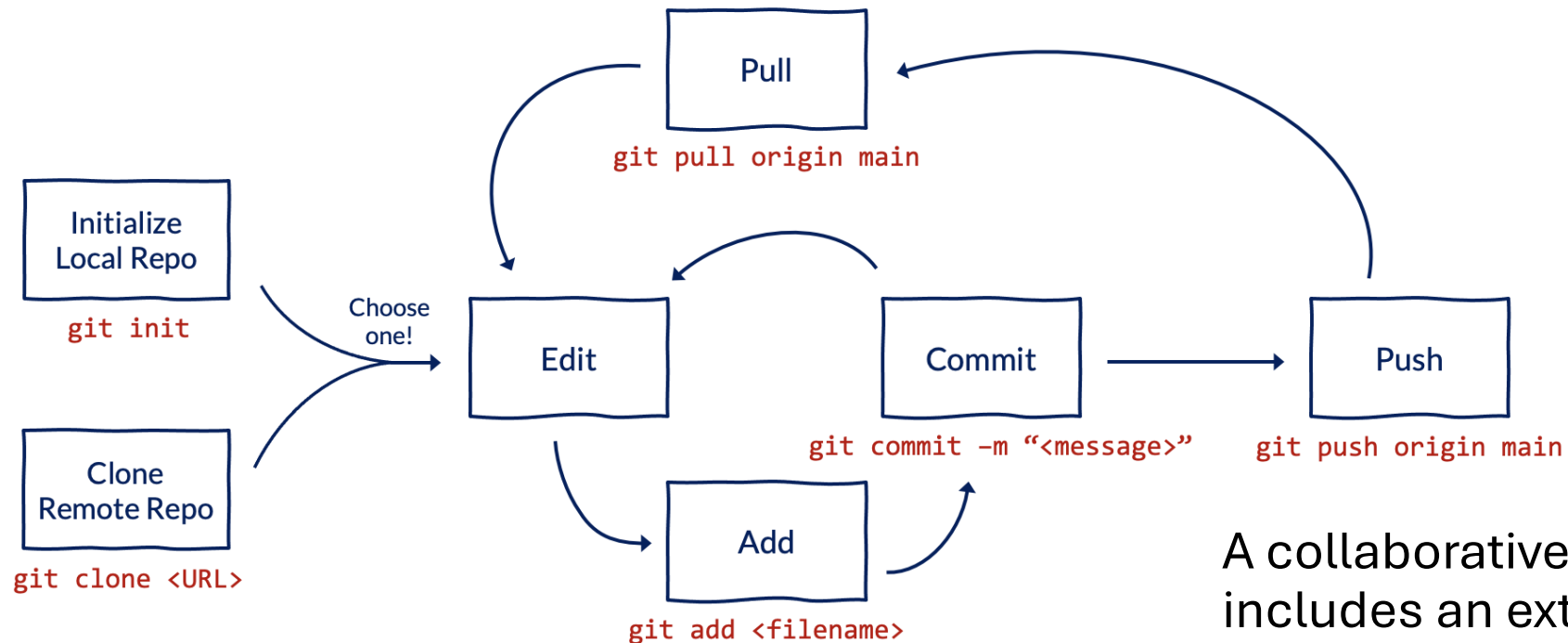
Demo: Part 1

- Create a new repository
- Clone an existing repository
- Edit a file on your local computer
- Add and commit the changes
- Push the updates to GitHub

Collaborative Workflow

LOCAL

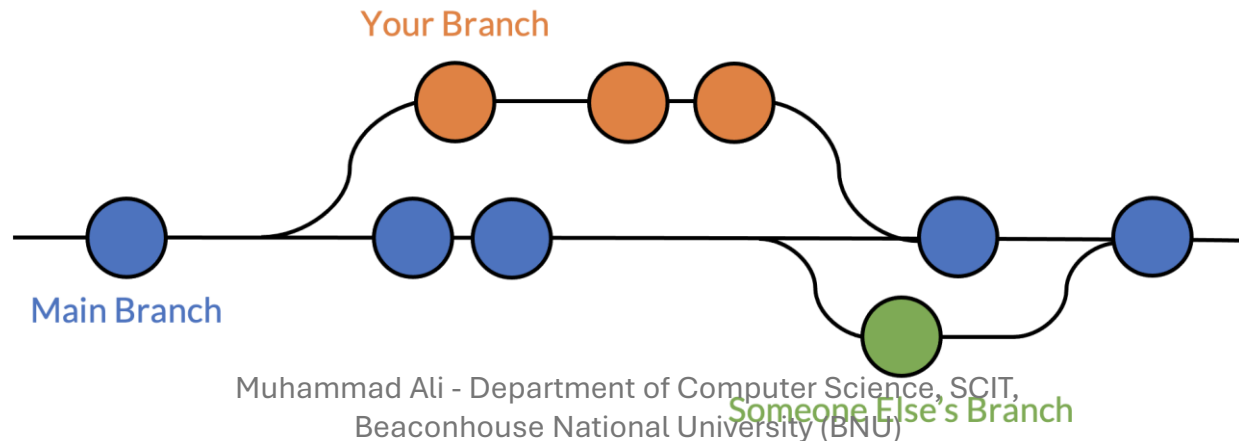
REMOTE



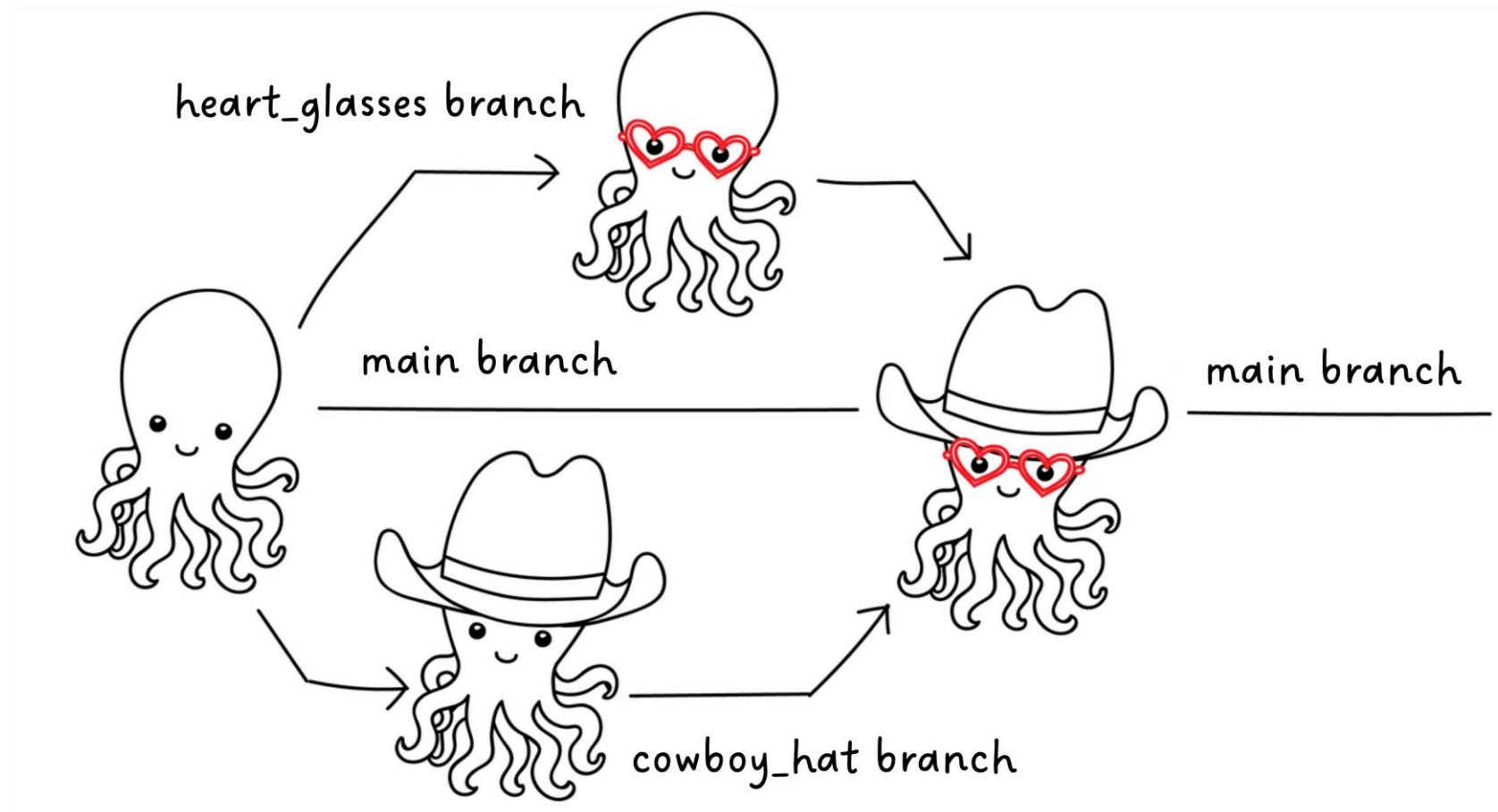
A collaborative workflow includes an extra step to **pull** updates from GitHub.

Collaborative Workflow - Branches

- **Branches** let you and your collaborators update existing code or add new code without fear of overwriting each others' files.
 - All commits live on a branch. The default branch in a repository is called the **main** branch.
- **Pull requests** let you **merge** changes from one branch into another.
 - GitHub provides a helpful interface to **review** pull requests.

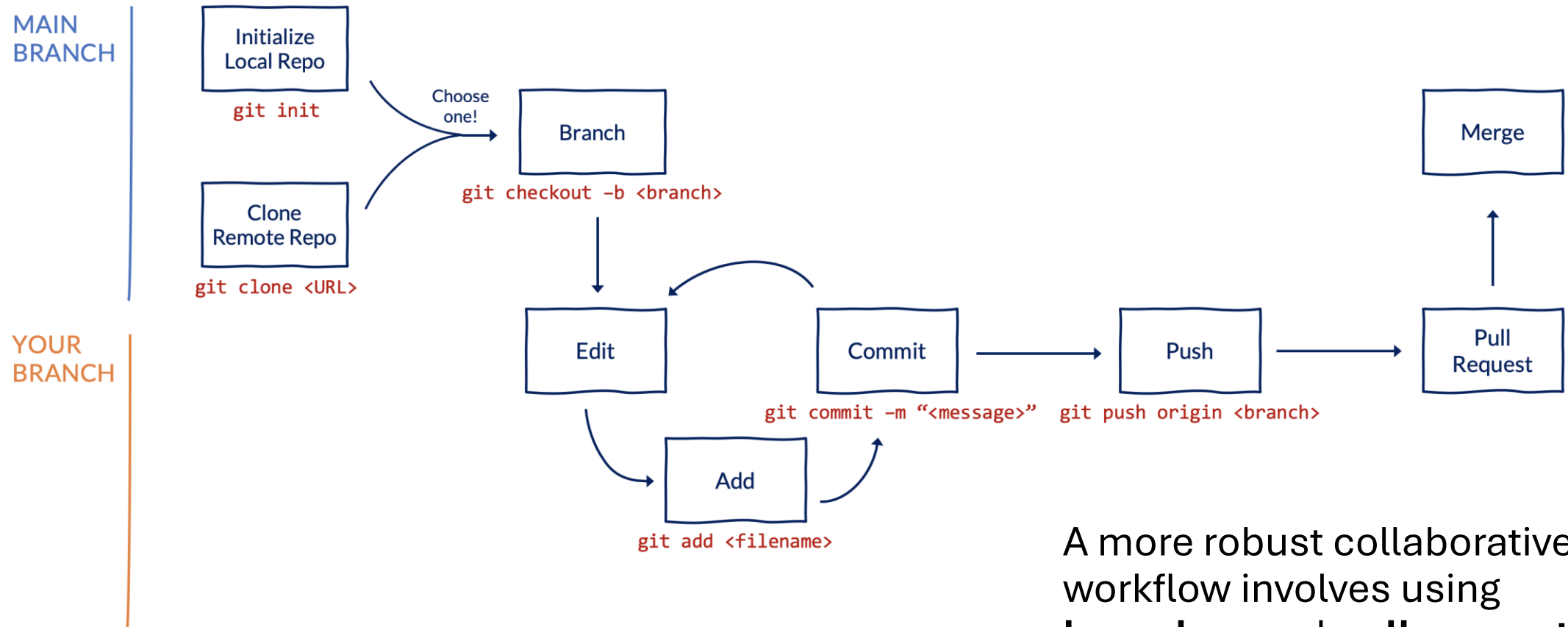


More Terms & Concepts



Branches let multiple collaborators make updates at the same time to later **merge** into the main branch.

Collaborative Workflow



A more robust collaborative workflow involves using **branches** and **pull requests**.

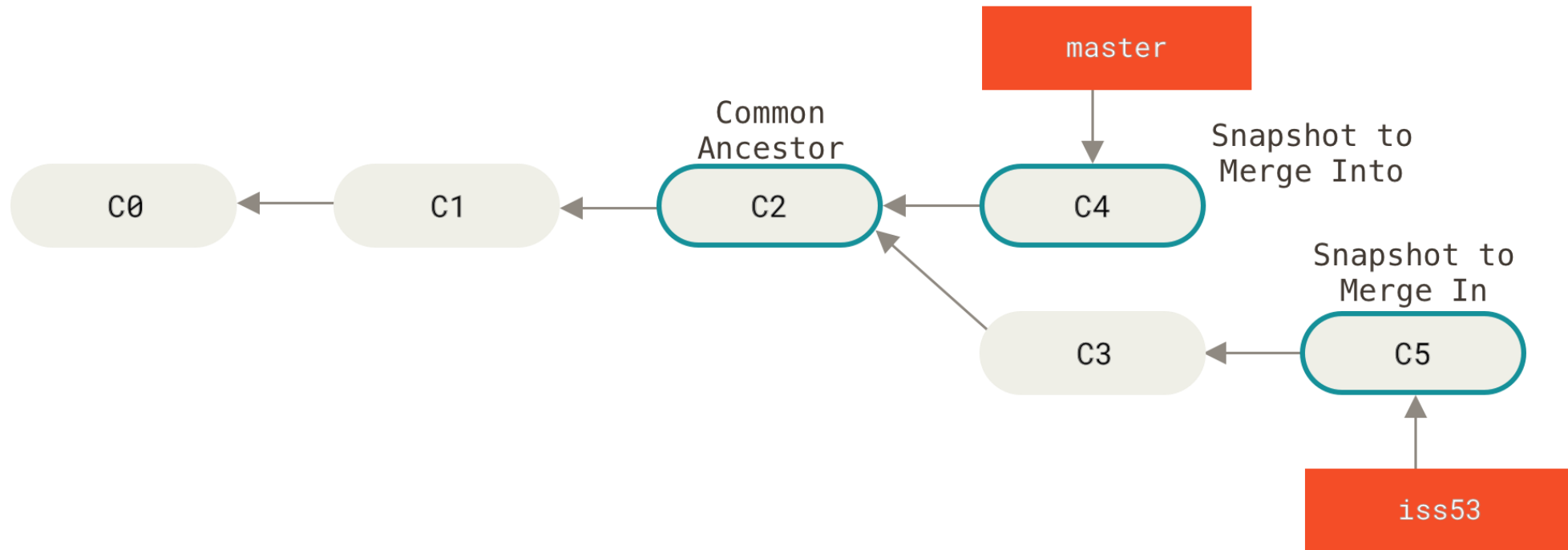
Demo: Part 2

- Pull updates from GitHub to your local computer
- Create a new branch
- Edit a file on your local computer
- Add, commit, and push the updates to GitHub
- Open a pull request
- Merge the pull request into the main branch

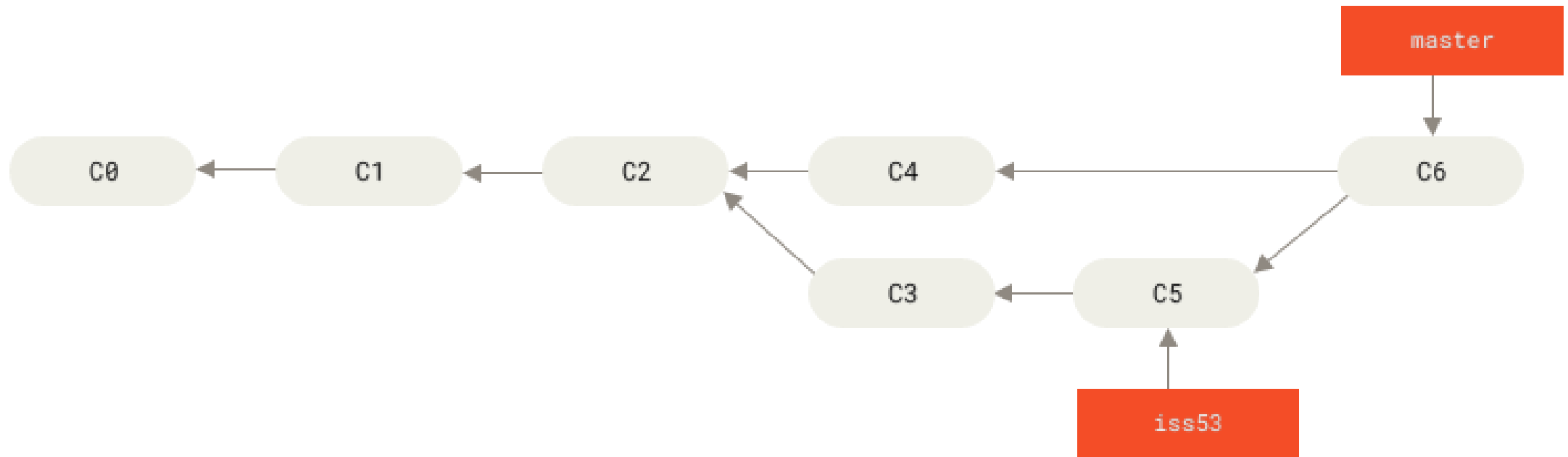
Rebasing

(merge, rebase)

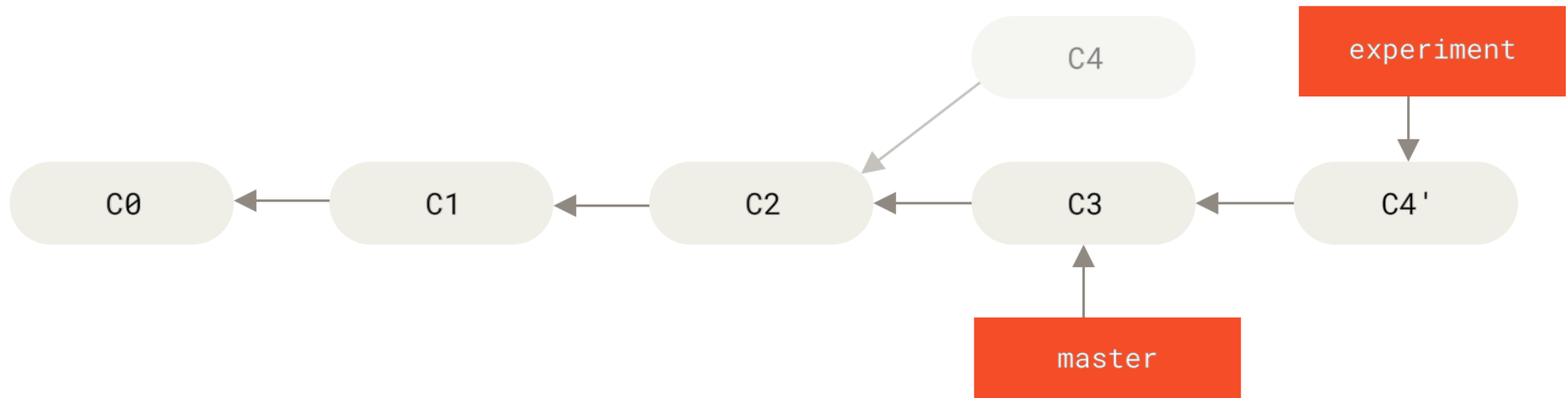
Git Merge



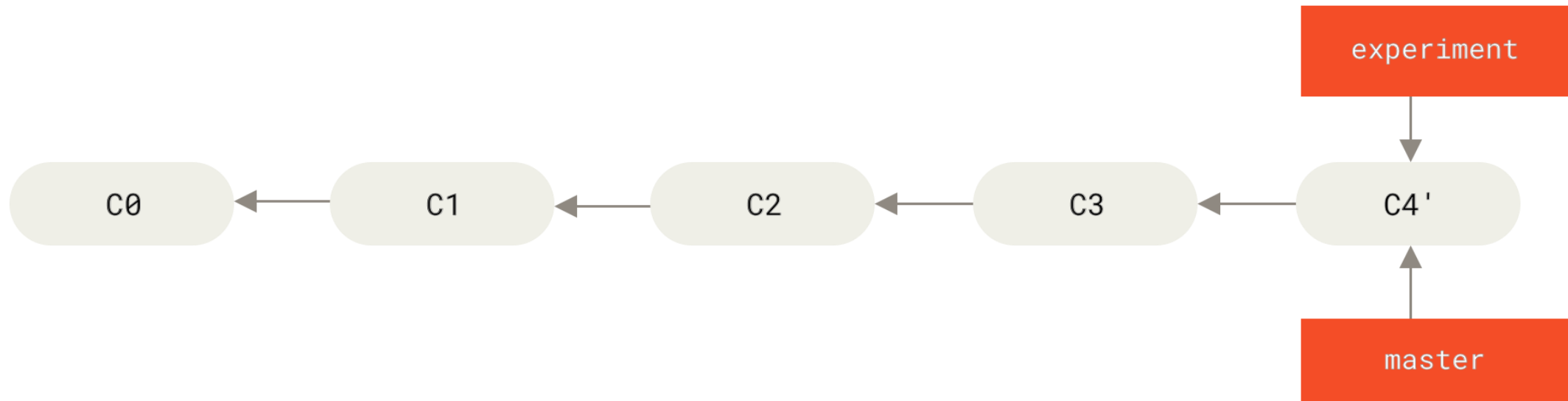
Git Merge



Git rebase

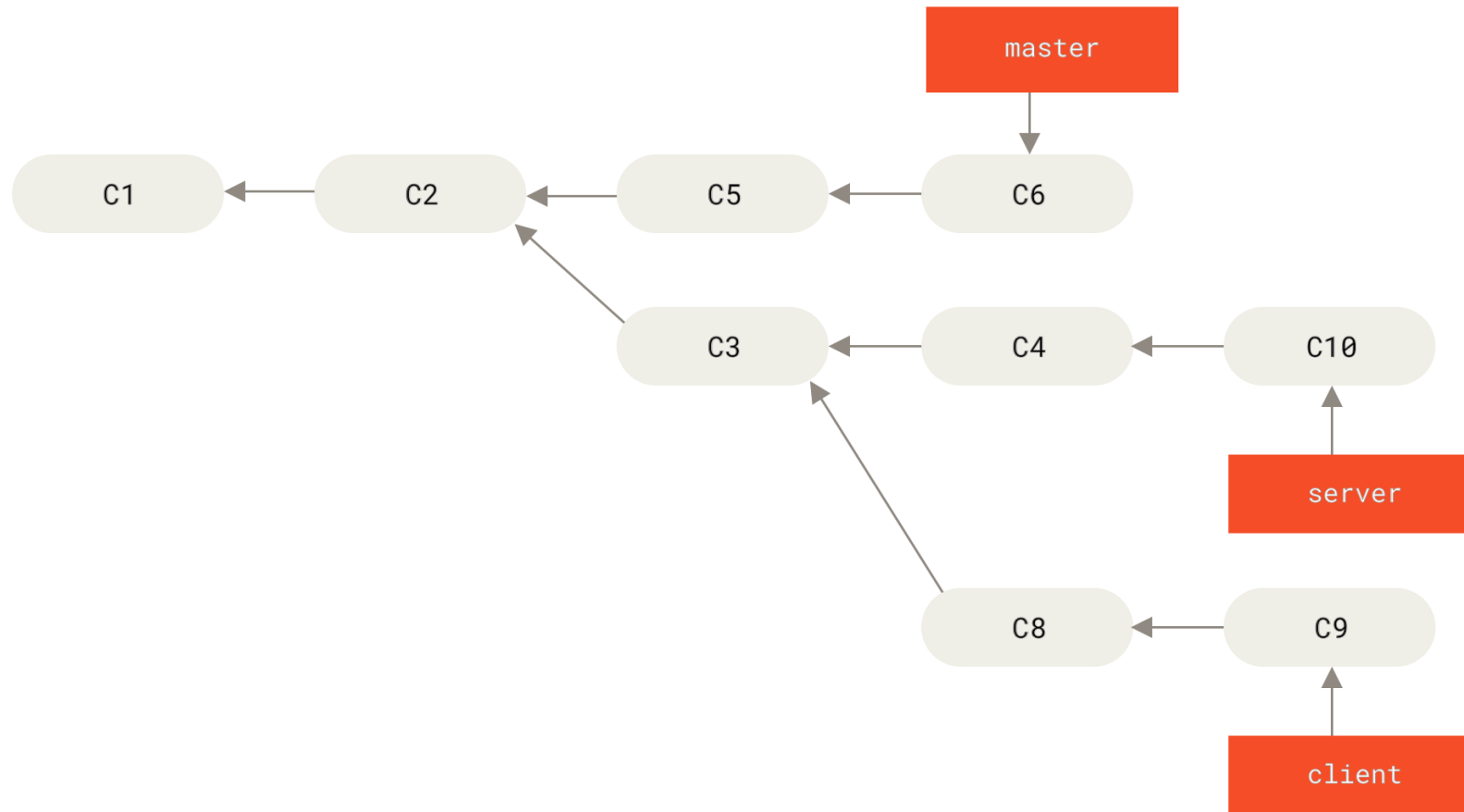


Git rebase

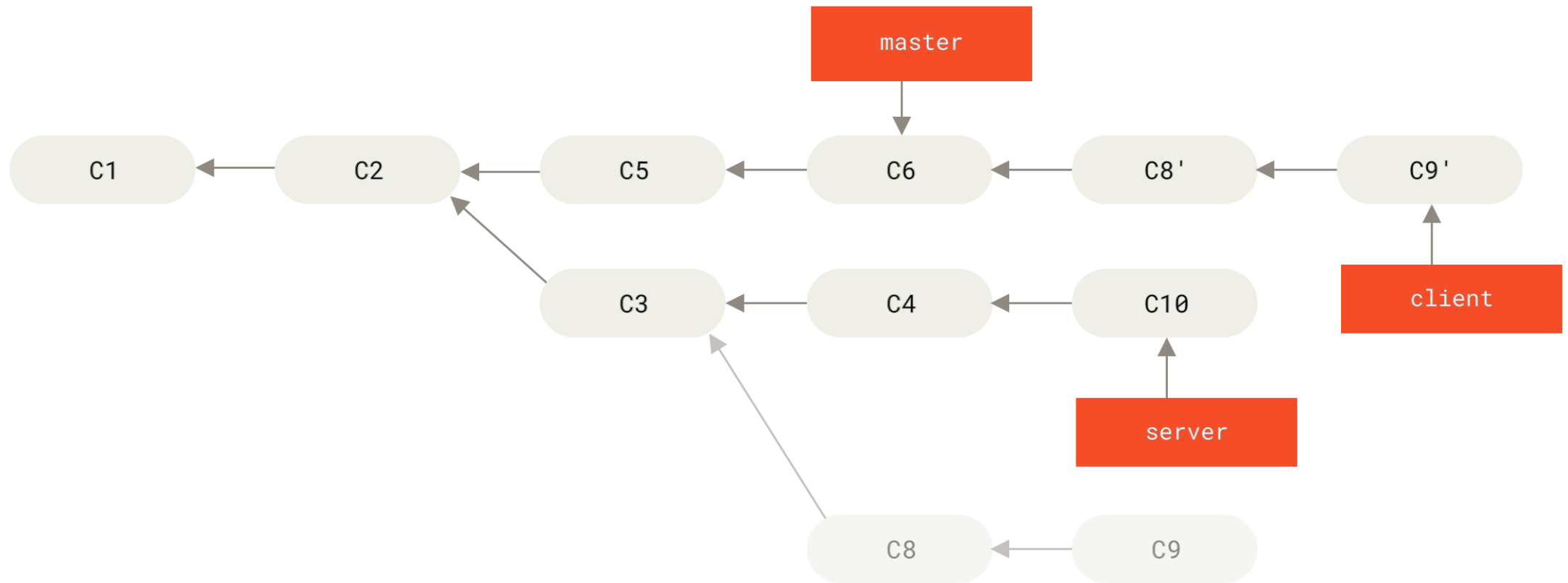


Rebase : More Interesting example

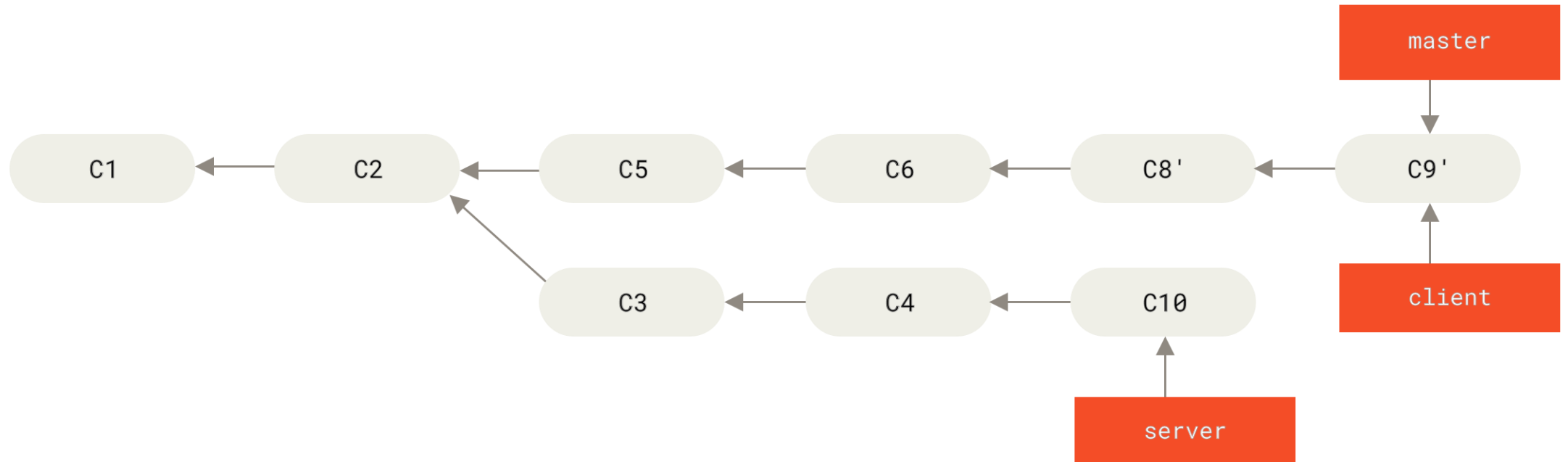
Git rebase



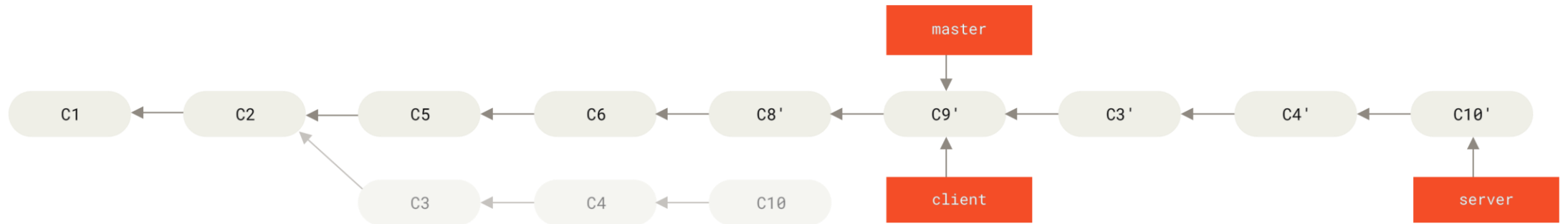
Git rebase



Git rebase



Git rebase



Undoing things

Undoing things in Git

restore:

a focused and safe way to **undo uncommitted changes** in your Git working directory or staging area

Unstage:

- `git restore <filename> or .` for all

Staged files

- `git restore --staged <filename> or .` for all files

Undoing things in Git

Using restore:

A) Undo changes in working directory (not staged)

- Restore file back to last commit:
- `git restore README.md`
- Restore **all** files:
- `git restore .`

B) Unstage changes (keep edits, remove from staging)

- `git restore --staged README.md`

C) Undo a commit but keep changes (rewrite last commit)

- If you committed too early:
- `git reset --soft HEAD~1`
- Now your changes are staged again.

Undoing things in Git

D) Undo a commit and discard changes (danger)

- `git reset --hard HEAD~1`

E) Undo a commit *without rewriting history* (best for shared branches)

- This creates a new commit that reverses an old one:
- `git revert <commit_hash>`
- Use revert when you already pushed and others might have pulled.

Key Commands

- `git clone <url>`: “Copy this remote project to my local computer.”
- `git add <filename>`: “Look at this file.”
- `git commit -m <message>`: “Take a snapshot of the files I’ve added.”
- `git status`: “Tell me what you’re looking at.”
- `git push`: “Send my snapshots to GitHub so others can see them.”
- `git pull`: “Replace my local version of this project with the remote version.”
- `git checkout -b <new_branch>`: “Create a new branch and switch to it.”
- `git checkout <existing_branch>`: “Switch to a different branch.”

Best Practices & Tips

- Always follow your organization's policies and avoid pushing potentially confidential information to GitHub.
 - In general, you should only track code on GitHub.
 - Consider whether a repository should be private or public.
- Use README files to document your repositories.
- Write descriptive commit messages to help your collaborators and future self.
- Work with your collaborators to develop workflows to meet your needs.
- Learn about other useful features from GitHub (e.g., [manage projects](#), [host websites](#), [automate tasks](#), and much more).

Next Steps

- These are some of our favorite resources to learn more:
 - [GitHub's Getting Started Guide](#)
 - [Happy Git and GitHub for the UseR](#)
 - [Getting Git Right](#)
- This is a great interactive exercise to practice what you learned today:
 - [GitHub Skills – Introduction to GitHub](#)